

Prague University of Economics and Business
Faculty of Informatics and Statistics



Matrix server implementation

BACHELOR THESIS

Academic program: B-AI Applied informatics

Author: Alexandr Belčenko

Supervisor: Bc. et Bc. Jaroslav Svoboda, MSc

Prague, May 2024

Acknowledgement

I would like to express my deepest gratitude to my supervisor, Bc. et Bc. Jaroslav Svoboda, MSc, for his assistance and guidance during the creation process. I would also like to thank my friends and family for their unwavering support.

Abstract

This work is primarily concerned with the open-source standard Matrix, developed in 2014 and popularised in the late 2010s, headed mainly by The Matrix.org Foundation as well as their for-profit cousin Element. The goal is to set up and configure a working instance of Synapse, one of the most popular servers built on the standard.

Initially, the Matrix protocol is introduced with a basic description of its history and fundamental concepts needed to understand its function. Then, there is a general overview of the current state of development of several matrix servers and clients, as well as some uses of the protocol notable for either their scale or the organisation behind them. Next comes the summary of the historical and current competition associated with the field.

Finally, there is a detailed description of the Synapse implementation itself, meant for a reader to potentially be able to follow it as an example for their own projects. This also includes basic statistics about the server's performance, requirements, and possible future improvements.

Keywords: Matrix, federation, home-server, open-source, Docker, Synapse

Abstrakt

Tato práce se zabývá především open-source standardem Matrix, který byl vyvinut v roce 2014 a zpopularizován koncem druhé poloviny desetiletí, veden především nadací The Matrix.org Foundation a od ní odštěpenou ziskovou složkou Element. Cílem je nastavit a nakonfigurovat funkční instanci Synapse, jednoho z nejpopulárnějších serverů postavených na tomto standardu.

Na začátku je představen protokol Matrix se stručným popisem jeho historie a klíčových pojmů potřebných k pochopení jeho funkce. Poté je uveden obecný přehled současného stavu vývoje několika serverů a klientů na něm založených a také některá použití protokolu pozoruhodná buď svým rozsahem, nebo organizací, která za nimi stojí. Dále je zde shrnuta historická a současná konkurence spojená s touto oblastí.

Nakonec je uveden podrobný popis samotné implementace Synapse, zpracovaný tak, aby se jej mohl případný čtenář řídit jako jedním z příkladů pro své vlastní projekty. Obsahuje také několik základních statistických údajů o výkonu a požadavcích serveru a případná budoucí zlepšení.

Klíčová slova: Matrix, federace, home-server, open-source, Docker, Synapse

Contents

Introduction	1
1 Matrix, history and function	3
1.1 History	3
1.2 Structure	3
1.3 Changes and governance	4
1.4 Basics	5
1.4.1 Server	6
1.4.2 Client	6
1.5 Communication	6
1.6 Federation	8
1.7 Bridges	8
1.8 Identity server	8
2 Current environment	11
2.1 Servers	11
2.1.1 Clients	12
2.1.2 Bridges	13
2.2 Notable uses of Matrix	13
2.2.1 Tchap	13
2.2.2 Germany	14
3 Competitors and predecessors	15
3.1 IRC	15
3.2 XMPP	15
3.3 Briar	16
3.4 Signal	16
3.5 Other	16
4 Setting up a home-server	19
4.1 Prerequisites	19
4.1.1 Docker	19
4.1.2 Hardware	19
4.1.3 SSL and reverse proxy	20
4.2 Setup	20
4.2.1 Docker Compose	23
4.2.2 Synapse configuration	23
4.3 Additional features	25
4.3.1 Email	25
4.3.2 Capcha	26

4.3.3	Endpoint restriction	26
4.3.4	Third party authentication	26
4.3.5	Automatically updating Docker containers	27
4.4	Performance	31
4.4.1	Methods	31
4.4.2	Hardware specifications	32
4.4.3	Small and large server comparison	32
4.4.4	Video and Audio call comparison	34
4.4.5	General resource usage	36
5	Future plans	39
5.1	Single Sign-On	39
5.2	Security	39
5.3	Orchestration	40
5.4	TURN	40
5.5	More detailed monitoring	41
	Conclusion	42
	References	45

List of Figures

1.1	Small diagram describing Matrix server architecture	5
1.2	An example of a JSON event representation	7
1.3	A diagram describing [matrix] bridges	9
2.1	Logo of Tchap	13
4.1	Command for generating the Synapse configuration	20
4.2	Initially generated sample YAML config file	21
4.3	Docker Compose YAML file	22
4.4	.well-known delegation file	24
4.5	Addition to Docker Compose YAML	28
4.6	Final configuration part 1	29
4.7	Final configuration part 2	30
4.8	Bash script used for gathering data	31
4.9	Docker stats JSON	32
4.10	Resource usage comparison between a small amount of traffic and joining a large server	33
4.11	Resource usage comparison between an audio and video call	35
4.12	Resource usage in 24 hours	37
5.1	Services supported by the official matrix.org server	39

List of Tables

2.1 Overview of server software 11

Introduction

The main goal of this thesis is the implementation of Synapse, a server based on the open standard Matrix, on a Linux server. A secondary goal is the description of the Matrix structure as well as the current environment and history surrounding its use.

Due to a portion of the classes the author has taken at the Prague University of Economics, in particular 'Information Systems Security' and 'Information and Communication Technology', he is already familiar with Linux and, specifically, Debian-like systems, as well as the implementation of software within this environment.

A later section required the gathering of raw data through measurements and its processing for the purpose of presenting an overview of statistics related to system resource usage. This employs the comparative method as well as several mathematical methods.

Since the visualisation of the data was later generated in the R programming language, this part also benefits from previous experiences.

The structure of this work begins with a general introduction to the Matrix protocol, with a summary of its history and how it came to be, followed by an explanation of its functionality and basic concepts needed to gain an understanding of how it works behind the scenes.

Afterwards, there is a general overview of currently available software based on Matrix, divided into the three categories of servers, clients and bridges. The following part then discusses the main previous and current competitors in the space.

The next section focuses on the implementation of Synapse on an Ubuntu server, with a description of the prerequisites needed for its implementation. All of the steps that were undertaken in order to install and configure the server are listed with the intention of a person with some understanding of the Linux architecture being able to use them as a jumping-off point to use in their own possible first implementation. At the very end of this chapter comes a collection of statistics gained from the server through performance monitoring. They include a comparison between high and low usage, video and audio calls and a general look at a 24-hour performance chart.

Finally, the last chapter lists possible future improvements to the project. It includes portions of the project that could have been accomplished differently or better. Some advanced functionality that would require more time in order to understand and implement, as well as some opportunities to possibly increase performance, like increasing redundancy and spreading the load over multiple servers.

The main benefit of this work is the final completed Synapse server, able to communicate within the Matrix network, users able to send and receive messages to and from any user and group also connected to the Matrix ecosystem with federation.

The project also serves to deepen the author's knowledge and experience in the field of messaging and networking, as well as work with remote servers altogether along with knowledge of Docker and Docker containers.

1. Matrix, history and function

Matrix is a relatively popular open-source protocol developed by a group of people now known as the Matrix.org Foundation for the purpose of instant messaging and communication.

It is mostly known as the technology behind the Element communications platform. Originally only used for instant messaging, it has grown to include several other technologies, namely Voice over IP (VoIP) and Internet of Things (IoT) communication.

1.1 History

Though today Matrix is mainly overseen by the Matrix foundation, this was not always the case. It was initially developed for an Israeli multinational software company named Amdocs and later spun off into a new subsidiary called Vector Creations Limited.

After 2017, Amdocs lost interest in funding the Project further. So, the core team behind Matrix decided to found a separate enterprise under the name New Vector Limited, which is currently trading as Element. In 2018, The Matrix.org Foundation was founded to serve as a neutral legal entity for further development of the standard (Matt Weinberger, 2014).

Today, it is one of the most popular ways to create public or private communication spaces for people looking to host their own end-to-end encrypted messaging server. However, a lot of the traffic is still centralised around the initial matrix.org server.

1.2 Structure

As defined by the Merriam-Webster online dictionary: *"protocol: a set of conventions governing the treatment and especially the formatting of data in an electronic communications system"* (2024).

Matrix is then better described as a set of rules for any implementation, client or server, to follow in order to be able to communicate with each other. The goal is to create a way to communicate that is easier to understand, secure and fully open (whether open-source or open for anyone to participate). A way to standardise this communication between different pieces of software, specifying all the different types of requests and responses a program can make, is called an application programming interface (API) (Jamie Juviler, 2024).

The different functions offered by services that are compliant with Matrix can be accessed on so-called endpoints. These are locations where a server receives requests associated with a specific service or resource. In Matrix's case, the API endpoints take the form of URLs, for example, `"/_matrix/media/v3/upload"` for uploading media or `"/_matrix/client/-`

v3/delete_devices” for deleting records of user devices. To interact with them, it is necessary to send an HTTP request containing the data being transferred to the relevant endpoint.

Several open APIs are named in the official specifications of Matrix intended for different types of applications needing to communicate with each other. Listed as they are there:

- client-server API
- server-server API
- application service API
- identity service API
- push gateway API

The first two’s purpose is apparent from their names, serving to communicate between a client and a server with other servers, respectively.

The application API is meant for external applications that modify and customise the server’s behaviour. This is mainly intended for developers wanting to implement some custom functions or automate specific tasks.

Its role is to enable monitoring of events like messages so that the application can interact with the server. The most common implementation using this API are bridges, discussed in section 1.7.

The identity API is used to connect with so-called identity servers that provide a way for the home-server to verify emails and phone numbers.

Finally, the push gateway API enables users to be notified with real-time push notifications about events occurring in a room they are participating in.

1.3 Changes and governance

Though an open standard does not definitively mean that it is open for changes by third parties, it is often the case that there is a way to add to the project in some form. For Matrix, this is usually done through the formal submission of a Matrix Spec Change (MSC) Proposal.

According to The Matrix.org Foundation (2024f) the current process is to create a markdown document describing all the changes and additions you wish to make and submit it to a specification proposal repository, where it is then debated over by members of the public as well as the members of the Spec Core team.

These are a team of people picked for their expertise in all of the different fields utilising Matrix that collectively act as a council to decide the future of the standard.

The discussion can, of course, also happen outside of the proposal repository. However, the

content of these outside discussions should still be summarised in the repository so that there is a future record.

After some time has passed, or if it seems like the proposal is ready and 75% of the SCT members agree, the Spec Core team can begin the final comment period. The following five days are dedicated to raising some final issues and objections that could cause the proposal not to pass. After it is over, the Spec Change Proposal is considered passed. The last necessary step is to submit the proposed changes with a working implementation example. This cannot then deviate too much from the original proposal. Otherwise, it is necessary to create a new Spec Change Proposal.

1.4 Basics

The basic system structure of the Matrix ecosystem can be broken up into two parts: the server and the client. While many applications work this way, Matrix is unique in that the vast amount of servers are decentralised, primarily serving a limited number of people. This is not to say that a larger number of users on a server is impossible, as shown by the number of users concentrated on the original matrix.org server.

Users who wish to chat with somebody have to join a room. A room can be either public or private, affecting whether people can join uninvited. Rooms can then be further grouped into Spaces, a relatively new change, making the system somewhat similar to the likes of Discord or Slack.

A room can also be encrypted or unencrypted, with options whether a guest can view the room before joining or whether new participants have permission to read the conversation history prior to their joining the room.

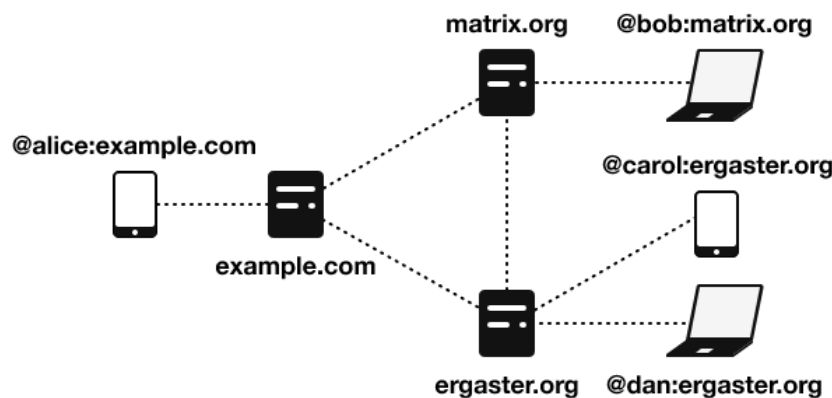


Figure 1.1: Small diagram describing Matrix server architecture
(source: <https://matrix.org/docs/matrix-concepts/elements-of-matrix/>) .

1.4.1 Server

Server software, in this case, called a home-server, runs locally on a machine accessible from the internet. Usually, that means it has its own public IP address and most likely a domain, though this is not a requirement since it is entirely possible and sometimes even recommended to operate solely within a private network, security concerns and similar troubles.

A user will then have a unique address combination, depending on which server they have registered under, the form of which usually looks like `@name_unique_on_server:servername.xyz`, as can be seen in figure 1.1.

The server is then responsible for any and all communication passing to other servers and its clients (The Matrix.org Foundation, 2023a). This can be viewed as similar to the SMTP architecture, with users being registered and using their own server while never connecting to other parties.

1.4.2 Client

Client software is required to access a server. It facilitates communication on behalf of the user and is, therefore, most likely to shape the user's opinion of the ecosystem. The client communicates with the server to receive or send messages. An important fact to remember is that *the clients themselves only ever talk to the home-server of the account they're using* (The Matrix.org Foundation, 2023a).

That means that the server handles all of the further communication. The client itself never has to connect to anything else. Of course, this also means that switching between servers is an arduous task, as it is almost impossible to automate and requires manually moving information like joined rooms one by one to the successor server.

1.5 Communication

There are many ways that data can be sent across the internet, and each has its advantages and disadvantages; for example, one of the main Matrix competitors, XMPP, uses XML to describe its messages.

On the other hand, Matrix communication is based on *events*, always represented with JSON objects. Every time someone sends a message, changes their name, or deletes a message, this information is converted into JSON format and sent to all participants.

Figure 1.2 demonstrates an example response from the server `rabbit.rocks` to the request `"/_matrix/client/v3/admin/whois/peter"` using the matrix Client-Server API to request information about the user "peter".

```

1 {
2   "devices": {
3     "teapot": {
4       "sessions": [
5         {
6           "connections": [
7             {
8               "ip": "127.0.0.1",
9               "last_seen": 1411996332123,
10              "user_agent": "curl/7.31.0-DEV"
11            },
12            {
13              "ip": "10.0.0.2",
14              "last_seen": 1411996332123,
15              "user_agent": "Mozilla/5.0 (X11; Linux x86_64)
16                ↪ AppleWebKit/537.36 (KHTML, like Gecko)
17                ↪ Chrome/37.0.2062.120 Safari/537.36"
18            }
19          ]
20        }
21      ],
22      "user_id": "@peter:rabbit.rocks"
23    }

```

Figure 1.2: An example of a JSON event representation

(source:

https://spec.matrix.org/v1.10/client-server-api/#get_matrixclientv3adminwhoisuserid)

Links between servers are usually protected by HTTPS, assuming that a server with a dedicated DNS domain also has an SSL/TLS certificate. The user messages are then optionally protected by end-to-end encryption, which is turned on by default.

User public keys are published to their respective home-server, where they can then be shared with any other server that requests to communicate with the user. The algorithm selected for device keys is Ed25519. This key never leaves the device it is associated with.

In addition to the device keys, which are long-lived, some encryption algorithms require that devices may also have a number of one-time keys, which are only used once and discarded after use. (The Matrix.org Foundation, 2024c)

Due to the users having these private keys, all their encrypted messages must be decrypted after each login, which may lead to long delays when loading a larger amount of messages.

1.6 Federation

When discussing federation in a networking context, it most often means two different entities operating as if they were one. Matrix uses this concept so that no matter which server a user is registered on, they can access and chat with any other user on any other server that is also open to federation. This uses the Server-Server API, specifically the `/_matrix/federation` endpoint.

In practice, this means that each server with at least one user participating in a room must keep a local copy of that room, which is updated with every change. This can get very complicated depending on the number of people in a room and the number of different servers having to communicate with each other. When joining a large room for the first time, the user may need to wait for up to an hour for the server to synchronise with the other participants since this requires reaching out to each and every server in the room.

1.7 Bridges

Bridges are a third part of this ecosystem. Although they are not strictly necessary, they are a helpful tool that allows access to the application they are made for. A bridge is an often tiny piece of code that translates requests between Matrix and services like Slack, MS Teams, or XMPP servers.

*Users on these communities appear as native users on Matrix, and ideally, the other way around on the third-party platform as well. The users created on the Matrix side by the bridge to mimic users on the third-party platform are called **ghosts**. The users created on the third-party platform by the bridge to mimic Matrix users are called **puppets** (The Matrix.org Foundation, 2023a).*

The original intention for Matrix could have been to only act as a glue that holds many different social platforms together and allows for interoperability between them, never actually to have so many users connected directly to it.

1.8 Identity server

A type of server already mentioned within the API explanations, identity servers are places that hold the references for the user's external identity, be it an email or a phone number. Using this, a user can be found by simply searching through the available entries for a matching identification method. In this way, it acts as a sort of address book, enabling users to be located across the network more easily. It also serves to validate that a specific user owns their email address and prevents imitation by others. The use of an Identity server is not mandatory.

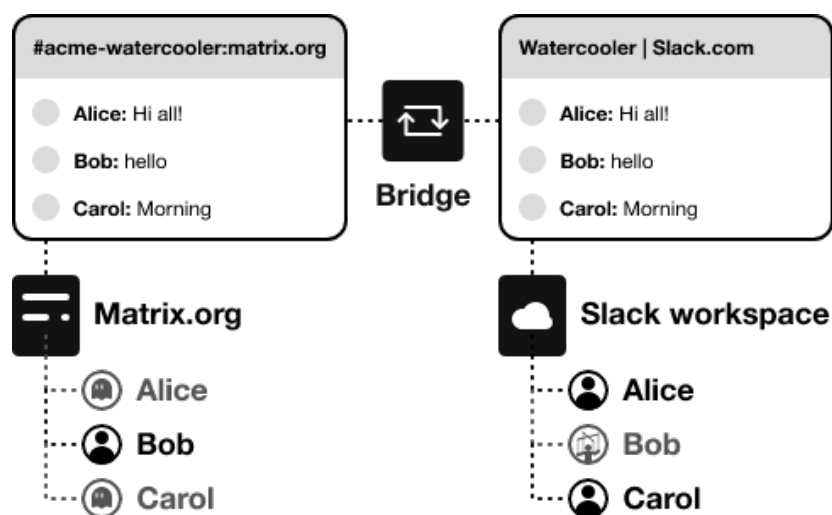


Figure 1.3: A diagram describing [matrix] bridges
(source: <https://matrix.org/docs/matrix-concepts/elements-of-matrix/>)

2. Current environment

While it is certainly possible to make a custom home-server and client from scratch, doing so would be very time-consuming and lead to vulnerabilities and errors. So unless one needs very specific features that are not found anywhere else or requires something for internal company/government use, the more likely alternative would be to use some of the already developed software in circulation.

Most of the presently listed software is thankfully open-source and can be used freely, even for commercial purposes, though specific licenses may have some restrictions not mentioned here.

2.1 Servers

According to The Matrix.org Foundation there are, at the time of writing, five server projects not marked as obsolete, all listed on their server selection page. The key difference between them is the number of features they support and the programming language they use as their basis.

Name	Status	License	Language
Synapse	Stable	AGPL 3.0	Python
Dendrite	Beta	Apache 2.0	Go
Construct	Beta	BSD	C++
Conduit	Beta	Apache 2.0	Rust
Telodendria	Alpha	MIT	C

Table 2.1: Overview of server software
(source: <https://matrix.org/ecosystem/servers/>)

The only server currently marked as "stable" is Synapse. This is because it was developed by the original team behind the Matrix protocol, so it was developed first. The big weakness of Synapse is that it was unfortunately written in Python, which, while it is a fine language, is not particularly known for its efficient use of system resources. A relatively recent event that can also be viewed as a disadvantage for Synapse is the change of their license from an Apache v2.0 to AGPL v3 following the repository ownership transfer to Element (Simmons, 2023).

It was a controversial change, due to this license using a Contributor License Agreement (CLA) that dictates all modifications to the application by third parties have to be open-sourced, which means that commercialising an application based on Synapse would become quite difficult, as any and all additions that could be developed for such an application to gain a competitive advantage must be available for free to the public.

Dendrite would be the obvious alternative since it also has mostly the same team of developers behind it but distinguishes itself by being written in the much more performant Go programming language, and while it doesn't yet support all of the features Synapse has, it is getting closer by the day. Since it is still in beta it is not yet recommended for general use. According to the same announcement by Simmons, Dendrite will also be moving under Element with the same restrictive license. However, compared to Synapse, the changes seem not to have been implemented yet, and the effects of this on the project are unclear so far.

Another interesting project and an alternative is Conduit, a server written in Rust. It probably has the most active contributors currently for a project not directly involved with the original Matrix creators. Though it is still quite a bit away from being on par with Synapse, it is still a valid choice for hobbyists and tinkerers.

As for the other two, Telodendria is still relatively early in the development process, and Construct has not really been actively developed for some time. Though some activity may be going on in the background, the only metric to measure it is to look at the frequency of repository contributions, of which, at the time of writing, there has been little, with the last commit being on May 1st, 2023 (jevolk, 2023).

2.1.1 Clients

Compared to the previous section, there are many more clients than there are servers because their development is proportionally easier, having to mostly only ever communicate with a single server at a time. Due to the sheer number of different implementations, there is little reason to list them all, so a select few of the more popular ones were chosen and will be described a bit further.

Element, made by the eponymously named company mentioned in the previous chapter, is perhaps the most feature-complete client application due to the fact it was developed almost concurrently with Matrix itself, though at the time called Riot. It runs on most desktop and mobile platforms with an additional web app, supports most features implemented in Matrix servers, and is the most used client overall. It also has a newer alternative version in development named Element X, aiming to support recent changes in the Matrix specifications and generally improve performance.

As alternatives, there are FluffyChat and Cinny, with the former being mostly a mobile-focused client and the latter a simplified web app aiming for a more clean interface. Of these two, FluffyChat is the only one that supports VoIP, though it struggles when running as a web app. One different client called SchildiChat also supports VoIP, but as it is a fork of Element, this is to be expected.

2.1.2 Bridges

Bridges are available for most popular instant messaging applications, from Discord to WhatsApp to even LinkedIn, though the MS Teams bridge, for example, requires a per-user subscription. If a particular application is missing, making a custom bridge with some programming knowledge is always possible since it is not a particularly complicated or large piece of software. The full list of currently available bridges is listed on the Matrix.org Foundation's website.

2.2 Notable uses of Matrix

Since Matrix is an open standard, it has found some interesting uses over its time in existence. The use of open-source protocols allows for easier replacement and maintenance of systems intended for long-term use and helps prevent reliance on a single company. This, as well as being able to maintain a completely private network of servers that still allow for communication within its borders, makes Matrix an attractive solution for governments and other public sector organisations, and indeed, there has been a history of successful projects in this sphere.

2.2.1 Tchap

The most notable example would be the Tchap project started by the French government agency DINUM (Interministerial Digital Directorate) in cooperation with ANSSI (National Agency for Information System Security) to *improve information dissemination and ensure the security of the French government communication network* (Dussutour, 2021).

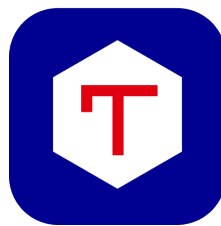


Figure 2.1: Logo of Tchap

(source: <https://www.tchap.gouv.fr/themes/tchap/img/logos/tchap-logo.svg>)

The open-source application, based on the then-still-named Riot, was developed in the late 2010s and launched in 2019 for testing in select governmental bodies and has been slowly making its way through the public sector. Due to being a federated network, it is simple to add new divisions by simply bringing a new home-server online and connecting it to the network (Element, 2024b).

Despite some early problems, like hacking attempts and other discovered security vulnerabilities, the last reported usage statistics from Metabase (2024) claim that it has about 530 thousand users, so the project seems to have been largely successful in its goals.

2.2.2 Germany

A similar project has also been realised in Germany, where a modified version of Element called BwMessenger has been deployed for use by the German armed forces since 2020, again mostly citing security concerns associated with using proprietary solutions and needing to host data locally (Steve Loynes, 2020).

The same, mostly government-owned company also developed a version for use by the wider public sector called BundesMessenger with modifications to make it simpler and faster to deploy. Similarly to Tchap, it allows for closer communication between different governmental bodies, which is important for a country divided into many relatively independent parts like Germany (Thévenet, 2023).

Germany, in particular, actually has many more such projects, like, for example, TI-Messenger, an open standard based on Matrix unifying Healthcare communication or openDesk, a comprehensive open-source product suite akin to something like Microsoft Office. Matrix is also used in many German universities and other research institutes.

3. Competitors and predecessors

Instant messaging (IM), in general, is quite an old concept, and the ways people have tried to standardise it are long and varied. In the last decade, many new communication protocols have been developed. This chapter will discuss a few protocols relevant to the idea of and competition with Matrix

3.1 IRC

Internet Relay Chat (IRC), is an old instant messaging system created in 1988 inspired by the Bulletin Board systems before it. Over the years it has been changed and improved by many different sources. Due to its long existence, many implementations have varying adherence to the standard. IRC is still widely used, though its popularity has waned. Modern IRC is understandably quite different from the past, having tried to implement many new features with varying degrees of success (Daniel Stenberg, 2021).

3.2 XMPP

As mentioned previously, XMPP, or the Extensible Messaging and Presence Protocol, is the largest direct competitor to Matrix, having been developed in the early 2000s under the name Jabber. The official Matrix specifications even mention it by name along with IRC when stating they wish to take their best features and avoid their mistakes.

XMPP is also a decentralised messaging protocol with independent actors running their own server instances, with various possible server and client applications developed over the years. Matrix and XMPP can even communicate with one another via bridge. Due to many XMPP servers having been in development for a much longer time, they can usually be counted on to take up a smaller chunk of resources than, for example, Synapse.

Another difference is the method of communication being XML rather than JSON, with this being a sore striking point to some. In reality, however, the difference in speed and resource usage between these 2 languages, when using high-performance parsers, is minimal, with the processing time for both being very similar (Lee, 2013).

An interesting comparison could be made with XMPP users being generally more spread out over a larger amount of servers, compared to the Matrix users being based mostly out of matrix.org. This could be due to XMPP simply not having such an initial server or maybe simply a much larger proportion of users being willing to self-host their instances. It also has to be said that XMPP is more than a decade older than Matrix, and therefore, many more implementations exist that have had the time to iterate and optimize.

3.3 Briar

Being released in 2018 Briar is one of the newest open-source communication protocols. Except compared to a normal stanrd it has no servers at all. It works completely peer-to-peer with everything being handled only directly through the device itself, trying to be as secure as possible. *If the Internet's down, Briar can sync via Bluetooth, Wi-Fi or memory cards, keeping the information flowing in a crisis. If the Internet's up, Briar can sync via the Tor network, protecting users and their relationships from surveillance* (Michael Rogers, 2024).

The main target customers of Briar are political dissidents, activists, journalists and most likely criminals, though this is of course not mentioned in their promotional materials. However, the concept of being able to communicate even without the internet is quite promising.

One of the consequences of this privacy is isolation, as no other apps exist that currently support Briar except its own. Though maybe that is the point of the matter, as if the communication escaped somewhere beyond this closed loop it would most likely compromise the privacy aspect of the app.

3.4 Signal

When talking about privacy-oriented IM apps, the name Signal usually comes up at least once. Released at about the same time as Matrix, it has become one of the most popular applications in its category. It even had federation for a few years, although only between a few servers and abandoned it after it was decided that federation was holding back innovation too much (Jake Edge, 2016).

Signal also has its own messaging and encryption protocols and is run by a non-profit entity. It relies on centralised servers but is also completely open-source. The only proprietary technology it uses is Google Cloud Messaging (GCM) since it is required to be able to use the servers. Signal is trying to hit a sort of niche between the security-conscious and the casual user. The app is user-friendly and does not require much setup, but also provides an acceptable level of privacy. Even with the GCM situation, the app is designed to work without it if necessary.

3.5 Other

One very popular application not mentioned here is Telegram, whose clients are mostly open-source as well. As for the others, almost every popular chat app has its own proprietary communications protocol, for example Discord, Teams or Slack. Each of these has several

times more users, whether because every company owns an MS 365 subscription or simply because of the network effect.

A possible reason why applications based on open protocols like Matrix and XMPP lag behind so much in users compared to the others is the ease of use. Signing up to an XMPP server requires more technical knowledge than many of their competitors, so it is only a minority of people who go through it. With Element this experience is better having a relatively modern client, but it is still missing features common in many other apps. One could even argue it is constrained by the need to adhere to a specification, though even that has some positives. It is interesting, however, how much it has grown in the government sector, so perhaps that is supposed to be the ideal use case.

4. Setting up a home-server

This chapter will show the implementation of the Synapse server, with the necessary preparations, the basic setup, some possible additional modifications and a final resource usage report.

4.1 Prerequisites

There are many platforms that Synapse supports, most of them based on the original Linux kernel, like Debian or OpenSUSE though some exceptions apply, like FreeBSD and OpenBSD. Windows server is, unfortunately, not supported, so it is impossible to run on there directly. This can be bypassed, however, with the combination of Hyper-V and Docker or the Windows Subsystem for Linux to effectively imitate a Linux server. An Ubuntu 22.04 LTS Minimal Server was made available for this work.

4.1.1 Docker

The official Docker container for Synapse was chosen as the installation method for this project, so Docker will need to be installed. For Ubuntu, it is convenient to refer to the official manual on <https://docs.docker.com/engine/install/ubuntu/>, and follow the instructions for installing the Docker apt repository. This also installs the docker-compose plugin seen in section 4.2.1. Similar instructions can also be found for other popular distributions.

Docker containers are similar to virtual machines, as they both provide isolated environments that contain applications, and the source system is more secure than when directly installing the software. A key difference between containers and VMs is that containers share a single operating system kernel with its host, reducing their size and allowing them to better cooperate with the outside machine.

This does limit them to only running applications made for their operating system, so only Linux applications on Linux and the same for Windows.

Due to being relatively lightweight, they are often used for deploying software to new systems quickly or containerizing software and using external tools to scale the number of instances based on performance.

4.1.2 Hardware

As for Hardware requirements to run Synapse, they vary quite a bit depending on the intended use case. The requirements for a small-scale server, without its users joining or creating

particularly large rooms, are quite low, needing less than 1GB of RAM. Larger rooms, however, require significantly more memory, especially when involving users from many different servers.

The server also requires a database to store its data, which can run alongside it on the same machine or remotely. A system resource usage and specifications breakdown can be seen in section 4.4.

4.1.3 SSL and reverse proxy

An SSL/TLS certificate is almost mandatory when wanting to federate with other servers since most will refuse to communicate through HTTP. It is generally easy to obtain thanks to the proliferation of Let's Encrypt, an organisation that provides free short-term certificates, the only requirement being an open port 80. In this case, though, a certificate had to be acquired from the Czech association CESNET through the university process to avoid opening said port to the broader internet.

To serve that certificate, it is recommended that a reverse proxy be used. Even though Synapse itself can also do so if needed, it is not the greatest choice for this task. Since Docker is being used, a reverse proxy is also the best way to serve traffic to the isolated Docker container, controlling which API endpoints the public can access. The normally recommended reverse proxy is Nginx, but Apache was chosen due to of prior experience.

4.2 Setup

As Docker has already been installed, the first step required is to run the command:

```
docker run -it --rm \
  --mount type=volume,src=synapse-data,dst=/data \
  -e SYNAPSE_SERVER_NAME=matrix.vse.cz \
  -e SYNAPSE_REPORT_STATS=yes \
  matrixdotorg/synapse:latest generate
```

Figure 4.1: Command for generating the Synapse configuration
(source: <https://hub.docker.com/r/matrixdotorg/synapse/>)

This generates a Docker volume, a sort of virtual storage space that contains files shared between the container and the host, usually having to do with configuration or logs. The generated volume contains a new configuration YAML file seen in figure 4.2 and a log configuration file. Both of which will be used later to configure Synapse.

An additional step to prevent a future problem is to create a directory named "media_store" in the newly created volume if it does not exist yet. All docker volumes are created in the /var/lib/docker/volumes/ directory, though it is possible to mount any local directory to a container if needed.

```
1 server_name: "matrix.vse.cz"
2 pid_file: /data/homeserver.pid
3 listeners:
4   - port: 8008
5     tls: false
6     type: http
7     x_forwarded: true
8     resources:
9       - names: [client, federation]
10       compress: false
11 database:
12   name: sqlite3
13   args:
14     database: /data/homeserver.db
15 log_config: "/data/my.matrix.host.log.config"
16 media_store_path: /data/media_store
17 registration_shared_secret: "lk_.qp&TK2gJuoJ=LfWtkm6:LY~BE1&6k5iN6;*@13."
18 report_stats: true
19 macaroon_secret_key: "j+^HHkNY#~xybgh:=Fj&o0v0f9o-D.ny1UsiXBU58U"
20 form_secret: "fH-P8=hzCqntCmtLvCI@t5Yg11axupsile*XtXOX8:r+OQ~TF7"
21 signing_key_path: "/data/my.matrix.host.signing.key"
22 trusted_key_servers:
23   - server_name: "matrix.org"
```

Figure 4.2: Initially generated sample YAML config file

(source: generated by script from <https://hub.docker.com/r/matrixdotorg/synapse/>)

```

1  synapse:
2      image: docker.io/matrixdotorg/synapse:latest
3      restart: unless-stopped
4      volumes:
5          - synapse-data:/data
6      depends_on:
7          - db
8      ports:
9          - '127.0.0.1:8008:8008/tcp'
10 db:
11     image: docker.io/postgres:alpine
12     restart: unless-stopped
13     environment:
14         - POSTGRES_USER=synapse
15         - POSTGRES_PASSWORD=<password>
16         - POSTGRES_INITDB_ARGS=--encoding='UTF8' --lc-collate='C' -lc-ctype
17 = 'C'
18     volumes:
19         - postgres-data:/var/lib/postgresql/data
20 volumes:
21     synapse-data:
22         external: true
23     postgres-data:

```

Figure 4.3: Docker Compose YAML file
(source: author)

4.2.1 Docker Compose

To start the container, it would be possible to use 'docker run' again with the necessary parameters. A more convenient way is a tool called Docker Compose. This tool also takes YAML configuration files and allows for the initialisation of multiple containers simultaneously without repeating the same 'run' command numerous times.

To go over figure 4.3, it comes in two containers, the first being Synapse and the second the database, here using Postgres.

The "image:" option specifies where to take the container from. The tag at the end indicates the version, with a popular one being ":latest" making it unnecessary to rewrite the configuration every time a new version of the container comes out. Of course, it is always good to check if updating the container to a newer version is safe and will not cause any conflicts.

The "restart" option has three possible values depending on whether to limit the number of restarts before a container gives up or restrict manually stopped containers from restarting.

The "volumes" option specifies the name of the volume to map to the inside of the container. In this case, it is also necessary to mark synapse-data as an external volume at the end of the document since it is already pre-generated using the previous command.

The "environment" option is used to pass parameters to the application that will be running inside the container. Here it is simply required that Postgres create a user for Synapse so that it can later use the database,

Finally, the "ports" option maps local ports to the container ports, which are needed for the server to be accessible to other applications on the machine. It will also be the port to which the reverse proxy forwards all Matrix-associated requests.

It is a good practice to test the configuration after finishing. The command to run the application stack is 'docker compose up -d' while being in the same directory as the Compose YAML file. The -d or --detach flag is to avoid being taken to the container console. The system will now download the necessary container images that aren't on the system yet and start the containers.

Listing all currently running containers can be done with the 'docker ps' or 'docker container ls' command. If the containers are listed as 'Up' and not 'Restarting', everything has most likely been configured successfully.

4.2.2 Synapse configuration

Everything to do with the Synapse configuration occurs inside of the homeserver.yaml file from the Docker volume.

The YAML configuration file shown in figure 4.2 is not complete yet. It is recommended to switch from the default SQLite database to a Postgres database as soon as possible. This is done by replacing information under the 'database:' tag. with the 'name:' for Postgres being psycopg2, the Python adapter for the database. The other data belongs in 'args:' with the required information being which user to connect to with which password as well as the name of the database, in this case inputting information selected from the Compose file, the host address and some other variables that can be seen on the final configuration file on figure 4.6.

```
1 {  
2 "m.server": "matrix.vse.cz:443"  
3 }
```

Figure 4.4: .well-known delegation file

(source: <https://spec.matrix.org/v1.10/server-server-api/#getwell-knownmatrixserver>)

The reverse proxy has to be configured to forward the endpoints `/_matrix` and `/_synapse/client` to the container so that the server can be accessed from the internet. In the case that it is impossible to expose ports 8008 and 8448, the default client and federation ports, the server has to have delegation configured to be able to communicate with external servers. This step applies here, since all ports except port 443 are unreachable outside of the university network.

A good way to do this is to serve a JSON response like in figure 4.4 through the reverse proxy under the address `/.well-known/matrix/server`. This way other servers can identify the port the server is reachable on. Like TLS, the Synapse server itself is also capable of serving this response if configured correctly, though this requires that all HTTP requests to the domain be routed to it, making it impossible to run Synapse on a separate subdomain.

Checking if federation is configured correctly is easily done with the Matrix federation tester located at <https://federationtester.matrix.org/>. If testing federation internally without exposing ports is necessary, building the tester package on a local machine and using the internal network is possible.

Any changes to the configuration require a restart of the Synapse server. This can be done with the 'docker restart container_name' command.

If everything functions correctly, the initial deployment can be considered successful. Adding users can now be done in two ways. First by setting `enable_registration` and `enable_registration_without_verification` to true or alternatively by accessing the container console, done using 'docker exec -it container_name bash' and using the `register_new_matrix_user` command. The latter method is preferred due to not opening up the server to anyone who wants to register without having the option to set up some protections first.

Any Matrix client should now be able to connect to the server and test out some basic functions. For specific clients refer to section 2.1.1.

4.3 Additional features

Enabling registration without any verification is regarded as an unsafe practice. Consequently, many options are available to secure the registration process and slow down any would-be intruders and other useful features that are not strictly necessary but nice to have. The final configuration can then be seen in figure 4.6 and 4.7.

4.3.1 Email

The simplest option is to enable verification via email. This requires access to an SMTP server that allows authentication by password. Unfortunately, most modern email providers now require OAuth2 authentication, making this more complicated. While it is possible to use a tool like Postfix, an SMTP Server or Relay, depending on the configuration, that can be configured to authenticate with email providers using OAuth2 access tokens, some email providers still allow for applications to use so-called "app passwords" which is more convenient while it is still available.

A new Gmail account was created for this purpose with the address vse.matrix.noreply@gmail.com and a newly generated app password. This does, unfortunately, limit the number of emails that can be sent to about 150-500 emails per day, though this limit can be increased up to 2000 with a Google Workspace account.

When first implementing the register with email functionality, many attempted configurations simply did not work, whether that be because of the SMTP server refusing to connect without OAuth2 or emails sent from the domain simply being filtered out by various spam filters and not arriving. The most likely option was something to do with the university reverse proxy blocking outgoing mail for most servers, this was however easily disproven.

A relatively simple way to test if the system is capable of sending email is through either the **telnet** or **openssl** command. Both can connect directly to a remote mail server where to send something with standard SMTP commands. An alternative can be found in **curl**, though this requires preparing all of the necessary text ahead of time. After attempting to test the connection with this method it appeared that this was not the problem.

What ended up being the main issue was a server misconfiguration. Synapse has 3 email settings associated with TLS:

1. `enable_tls`
2. `force_tls`
3. `require_transport_security`

While all of these options appear to do the same basic task, they are separate, and because all were set to true at the same time they were interfering with each other. 'force_tls' and 'enable_tls' are only needed in the case of connection to a server via port 465, connecting

implicitly through TLS. The other option is meant for connections via port 587 where the connection is then upgraded through STARTTLS after a request.

4.3.2 Capcha

Another good protection measure is a CAPTCHA, a small window that allows the server to send the user a small challenge to prove they are human. The full version of the acronym is quite long, but it can be reduced to the shorthand Automatic Turing test. A Turing test is meant to measure how close machines have come to successfully imitating humans. In the same vein, a CAPTCHA tries to filter out robots from accessing a website by analyzing user behaviour and giving them tasks that would be easy for a human yet very difficult for a computer to solve.

Given the advancements in Artificial intelligence, it is not yet clear how effective such measures will be in the future, and even today, these tests are easily bypassable by using cheap labour in developing economies, but it is still another possible layer to add that can deter or at least slow down attackers. It is also relatively easy to set up, requiring only a public and private ReCaptcha key, able to be created using any Google account at <https://www.google.com/recaptcha/admin/create>. The graphical version of the CAPTCHA is then the responsibility of the client to display since otherwise it will be impossible to register. The option associated with this topic is 'recaptcha_public_key' and 'recaptcha_private_key' which will then start requiring its completion upon registration.

4.3.3 Endpoint restriction

To restrict what users can and cannot register it is possible to restrict access to the `/_matrix/client/v3/register` API endpoint in the reverse proxy. This process can be done for any such endpoint exposed by the server to varying results. When attempting to restrict access to all possible interactions with the client-server API to be only available from the university network it led to federation breaking down completely, even though that functionality should be using a completely different API with different endpoints. It is important to be aware of at least what types of endpoints serve what general purpose in order to avoid such complications in the future.

4.3.4 Third party authentication

To quote Tom Scott: *"You shouldn't store passwords yourself if you can at all avoid it"* (Computerphile, 2013). The manner in which to achieve this goal can be varied, with the most common way being through third-party logins. Services like Facebook, Google and Apple provide a much safer environment where passwords can be stored and also move the responsibility of keeping passwords onto the other party.

To be able to log in using a third-party account, the functionality has to be configured from both sides. The protocol that allows for this functionality is OAuth2, or more specifically, in this case, an extension of it called OpenID Connect (OIDC). Synapse can be configured to use OIDC providers instead of or alongside its native password management system. Note that the library required by Synapse to do this is preinstalled within the Docker image, but needs to be added manually otherwise.

Most providers also offer a developer console for the user to be able to configure the relation from their side. A client id and a client secret must be generated and stored in the configuration. The server currently supports such accounts from Google and GitHub, with the former having their setup on the <https://console.cloud.google.com/> page and the latter on <https://github.com/settings/applications/new>. GitHub actually does not support the OIDC standard, instead being a regular OAuth2 provider, so the configuration is still similar but with certain changes (The Matrix.org Foundation, 2023c). The options associated with OIDC in Synapse are 'oidc_providers:' with each following identity provider being tagged as '- idp_id: provider_name'

4.3.5 Automatically updating Docker containers

Updating any software product to a new version is always quite risky. Depending on the willingness of the dev team, the software may be fully backwards compatible or it may break completely. As long as the risk is acknowledged it is often more comfortable or at least less tedious not to have to manually recreate containers and rather automate the task.

There are several ways to achieve this, depending on the size and importance of the server. For the purposes of this work, though, the open-source application Watchtower, intended mostly for hobbyists and other small-scale users, is sufficient.

How it works is that Watchtower checks each day whether the indicated container is out of date with the original, and if so, shuts down the container and replaces it with the new version running with the same parameters as before (simskij, 2023).

Since all of the configuration is already placed in the Compose YAML file, it is now easy to add a new application on top of that. To set up Watchtower now only requires adding the lines seen in figure 4.5 to the configuration file as well as adding container IDs to identify them and activating it all with 'docker compose up'.

```
watchtower:
  image: containrrr/watchtower
  container_name: watchtower
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  command: synapse-container postgres-container
```

Figure 4.5: Addition to Docker Compose YAML

```

1  server_name: "matrix.vse.cz"
2  pid_file: /data/homeserver.pid
3  public_baseurl: https://matrix.vse.cz/
4  listeners:
5    - port: 8008
6      tls: false
7      type: http
8      x_forwarded: true
9      resources:
10        - names: [client, federation]
11          compress: false
12  database:
13    name: psycopg2
14    args:
15      user: synapse
16      password: <password>
17      database: synapse
18      host: db
19      cp_min: 5
20      cp_max: 10
21
22  log_config: "/data/matrix.vse.cz.log.config"
23  media_store_path: /data/media_store
24  registration_shared_secret: "<secret>"
25  macaroon_secret_key: "<secret>"
26  form_secret: "<secret>"
27  signing_key_path: "/data/matrix.vse.cz.signing.key"
28  trusted_key_servers:
29    - server_name: "matrix.org"
30
31  email:
32    smtp_host: smtp.gmail.com
33    smtp_port: 587
34    smtp_user: "vse.matrix.noreply@gmail.com"
35    smtp_pass: "<password>"
36    require_transport_security: true
37    notif_from: "Matrix <vse.matrix.noreply@gmail.com>"
38    notif_for_new_users: false
39    validation_token_lifetime: 15m
40    invite_client_location: https://app.element.io

```

Figure 4.6: Final configuration part 1
(source: author)

```

41 enable_registration: true
42 registrations_require_3pid:
43   - email
44 recaptcha_public_key: 6LcsJaYpAAAAAN7HKQSeodRuWo8rvapaZZcV3dtp
45 recaptcha_private_key: <key>
46 enable_registration_captcha: true
47
48 allow_public_rooms_without_auth: false
49 allow_public_rooms_over_federation: true
50 include_profile_data_on_invite: false
51 require_auth_for_profile_requests: true
52 oidc_providers:
53   - idp_id: github
54     idp_name: Github
55     idp_brand: "github"
56     discover: false
57     issuer: "https://github.com/"
58     client_id: "<id>"
59     client_secret: "<secret>"
60     authorization_endpoint: "https://github.com/login/oauth/authorize"
61     token_endpoint: "https://github.com/login/oauth/access_token"
62     userinfo_endpoint: "https://api.github.com/user"
63     scopes: ["read:user"]
64     user_mapping_provider:
65       config:
66         subject_claim: "id"
67         localpart_template: "{{ user.login }}"
68         display_name_template: "{{ user.name }}"
69   - idp_id: google
70     idp_name: Google
71     idp_brand: "google"
72     issuer: "https://accounts.google.com/"
73     client_id: "<id>"
74     client_secret: "<secret>"
75     scopes: ["openid", "profile", "email"]
76     user_mapping_provider:
77       config:
78         localpart_template: "{{ user.given_name|lower }}"
79         display_name_template: "{{ user.name }}"
80         email_template: "{{ user.email }}"

```

Figure 4.7: Final configuration part 2
(source: author)

4.4 Performance

Due to the server running in a Docker container, the process to obtain the relevant raw data is relatively uncomplicated. The 'docker stats' command gives a general summary of resource usage statistics, such as CPU and memory usage, as well as the amount of network traffic arriving at and departing from the container. Containers can be queried separately or together.

```
for i in {1..360}; do  
docker stats —all —format json —no-stream >> data.json  
sleep 5  
done
```

Figure 4.8: Bash script used for gathering data
(source: author)

4.4.1 Methods

The method of gathering data used in this work is to write a simple bash script with the command shown in figure 4.8 that would print the telemetry information directly to a file and afterwards wait 5 seconds before repeating this again. To avoid a situation where the process would run indefinitely until corrected, the loop was given a limited amount of repetitions.

The fact that the 'docker stats' command allows for showing the statistics as a JSON object allows for the output to be easily parsed by most modern programming languages. An example JSON record recorded from the Docker container can be seen in figure 4.9. Unfortunately, due to the output still being in a human-readable but not useful format, there are still some steps left before being able to analyze the statistics. Using some simple regular expressions, mostly having to do with removing parentheses to make the variable interpretable as numeric instead of a string, and splitting up variables that contain multiple values, it is possible to reduce the file down to only necessary components.

With the help of the jsonlite library, the contents were then loaded into RStudio, a development environment for the R programming language, where it was then possible, with some changes, to analyse the data or simplify it in charts with the ggplot2 package.

An important note if one wishes to replicate this process: The CPU percentage that docker stats uses is based on the number of cores available to the CPU, meaning that if it has, for example, 4 cores, the maximum percentage docker stats can report is 400%. It is necessary to adjust the data to reflect this afterwards if required.

```

1 {
2   "BlockIO": "0B_/_0B",
3   "CPUPerc": "0.15%",
4   "Container": "710919919d3d",
5   "ID": "710919919d3d",
6   "MemPerc": "0.64%",
7   "MemUsage": "101.9MiB_/_15.58GiB",
8   "Name": "synapse-synapse-1",
9   "NetIO": "44.8MB_/_87.3MB",
10  "PIDs": "16"
11 }

```

Figure 4.9: Docker stats JSON

4.4.2 Hardware specifications

To add context to the following graphs it would be useful to list the specifications of the machine the server is running on. This being:

- Intel(R) Xeon(R) CPU E5-2420 v2 @ 2.20GHz
- 62 GB Storage
- 15 GB Memory

4.4.3 Small and large server comparison

Figure 4.10 shows a group of charts showing the difference in scale of resource usage between a server without major federated connections to a different server, and one on which a user has joined a server with approximately 600 members. Both sessions were monitored for 30 minutes.

The most interesting graph here is probably the CPU usage graph, as it clearly shows the initial period when joining a room. Because of the fact that multiple servers participate in the room, it is necessary to contact every one of them to notify them of the user's participation. What can also be seen is a little spike in usage shortly after 20 minutes in the test run. This period simulates active messaging activity between two users on different servers.

As for memory usage, a clear spike can be seen, though interestingly, only a few minutes after the initial spike in CPU usage. When joining large servers, it is important to keep track of available RAM because, as seen on the chart and confirmed through personal experience, the amount of memory a server only ever rises, never falls. Especially when connecting to a room with many different servers. What also increases is the network traffic, but that is to be expected when communicating with a large amount of other servers.

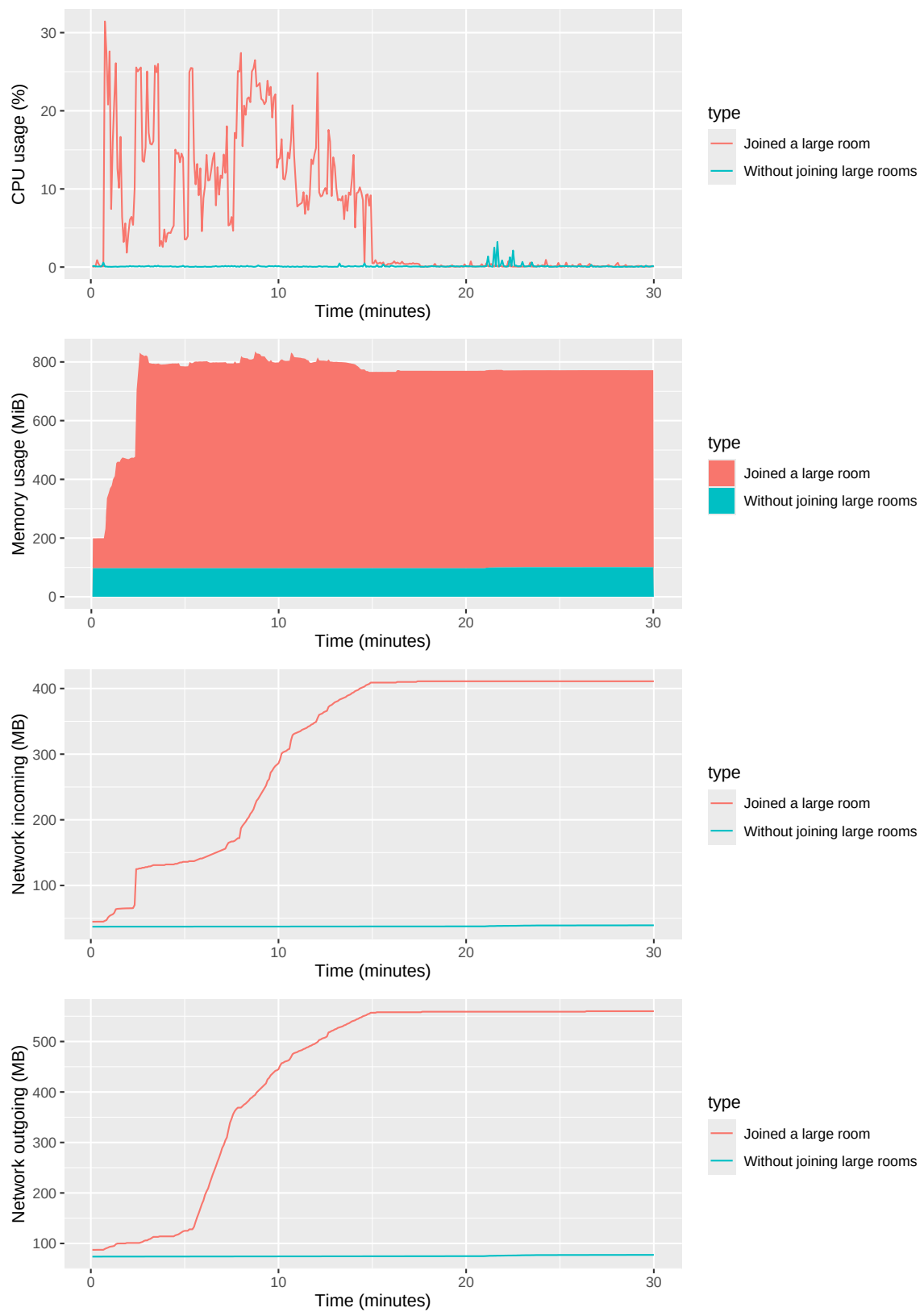


Figure 4.10: Resource usage comparison between a small amount of traffic and joining a large server

(source: author)

4.4.4 Video and Audio call comparison

The second set of charts visible in figure 4.11 is also a comparison, but this time it is between a video and an audio call, both having been recorded over the course of 10 minutes. Despite expectations, the difference in network throughput between a video and an audio call is quite small, though the video of course requires more it differs only by a few Megabytes.

Another anomaly appears when looking at the CPU usage graph. There seem to be random spikes of higher usage between longer periods of otherwise very low levels. The only speculative idea that comes to mind is that it has something to do with either buffering or these were the times when there was movement on camera. No other graph seems to show the same increase though.

Memory usage seems to be relatively high with video as expected, though it seems that the amount of memory used could be predetermined since the changes are so minuscule they almost can't be seen.

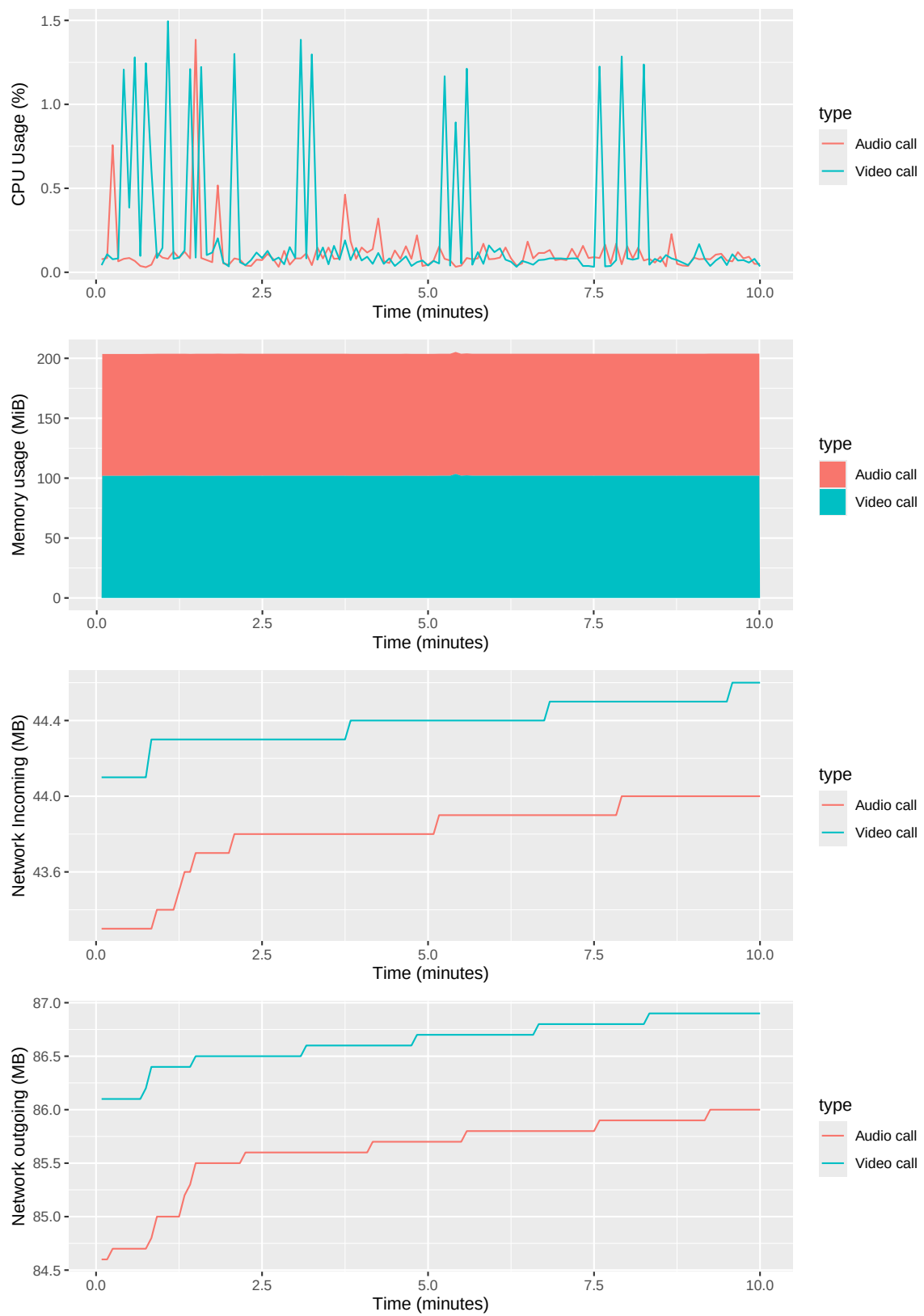


Figure 4.11: Resource usage comparison between an audio and video call
(source: author)

4.4.5 General resource usage

The final set of charts in figure 4.12 shows how the server performs more long term, with the measurement period being 24 hours. The user has joined another server, but this time the member count is approximately 10 thousand.

The same processor usage spikes can once again be seen on the first chart, though they represent much longer usage spikes compared to previously. The correlation between the increased CPU usage and the rise in memory usage can be seen quite clearly up until the 15-hour mark.

An interesting fact is that the memory usage seems to actually have gone down a bit when compared to the previous 30 minute test, even though the number of users has increased by a factor of 10. This can be either due to the need for memory naturally going down as the server only needs to keep the most recent messages in memory or that without many users interacting with it, the server uses a constant amount of RAM. What has once again risen, as predicted, is network usage. Since this statistic also includes communication with the database, this requires quite a large throughput.

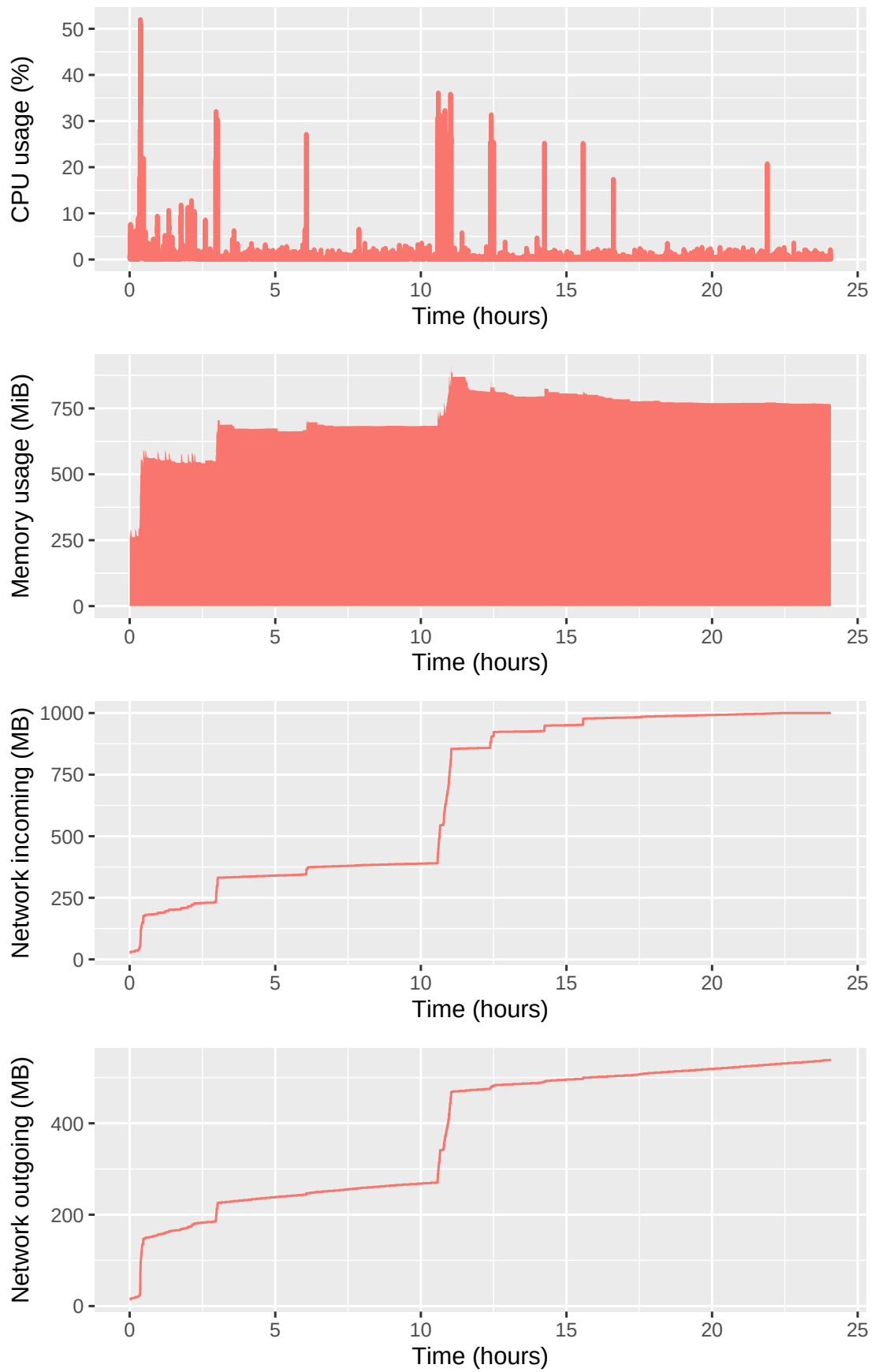


Figure 4.12: Resource usage in 24 hours
(source: author)

5. Future plans

It is always possible to do a task better than before. One cause is that projects, especially when having anything to do with modern technologies, have to constantly be updated and reworked as newer updates come out, and maintainers have to adapt or risk their instances containing severe security vulnerabilities, compromising the safety of their users. When doing anything for a second time, the knowledge gained from the initial attempt helps with being able to accomplish the task faster and seeing potential problems earlier.

This chapter will discuss possible future changes and improvements that could be added to this project to make it better.

5.1 Single Sign-On

Though the server does support Github and Google, this is far from the maximum amount of services Synapse can support. An example can be seen in figure 5.1, with the official server supporting 5 different popular services. Synapse, of course, supports password authentication protocols like LDAP or even more complex solutions like Shibboleth and many other open and closed-source OpenID providers, as well as others like the CAS protocol.



Figure 5.1: Services supported by the official matrix.org server

Depending on how the future of this project could play out it would be possible to maybe set up a new authentication service with free software like Authentik to use a wider collection of applications with only one account.

5.2 Security

The original intention was simply to restrict access to any endpoint used by the client API to be restricted to operate only within the Prague University of Economics network. Unfortunately, denying any other requests to the API led to the necessary functions of the client, such as the option to view a list of public rooms on a different server, breaking. The restrictions were, therefore, limited to only denying the endpoint for the login and register functions. With more time to experiment, it would be possible to make this restriction more fine-grained and useful.

A further improvement to the Security of the server would also be to use monitoring software

to scrape server logs and filter unwanted traffic. Synapse configuration also allows for specific IP addresses to be restricted from communicating with the server which could be combined with the previous option to manage access.

Due to using a new Gmail account in order to send confirmation emails, many SMTP servers will mark the email communication from it as spam or, even worse, not deliver the message at all, as is the case with the university mail server. An improvement would be using a more professional email service that would be excluded from these restrictions. If this were to be implemented, and emails were guaranteed to arrive to the recipient, it would be possible to start restricting the format of viable addresses so that they, for example, all come from the same domain, like the university or a personal email server.

5.3 Orchestration

Something that could be implemented is creating more instances of the central Matrix server and linking them up together. The technology used to do this in Docker is called Docker Swarm, which manages the deployment of multiple containers on multiple Docker hosts. Swarm isn't the only possible container orchestration tool, a more popular alternative being Kubernetes.

When used within a single server, it can be useful as a redundancy tool so that if one container encounters an issue, the other can take over until the first one recovers or is repaired; it isn't very helpful with performance, as both containers have access to the same resources.

It is with multiple servers that an orchestration tool is truly useful, as it can assist with the synchronisation between multiple machines while acting as one. Because these are separate machines, this makes it easier to scale the server depending on the number of users and the load it is under so that whenever it is overwhelmed with requests, the container manager can simply request that another instance be initialised on a separate machine.

5.4 TURN

A further improvement would be the implementation of a TURN server. TURN stands for Traversal Using Relays around NAT, and it helps connect two users in case their NAT or firewall blocks the direct connection between them. A TURN server would then step in and act as an intermediary between the two users, transferring the network packets between them.

5.5 More detailed monitoring

When considering different approaches toward tracking system resource usage for section 4.4 there was an alternative way to get the necessary information that required a more complicated setup. This was a program called Prometheus, which is also the recommended way to set up monitoring on a Synapse server. The server allows for the exposure of an additional `/_synapse/metrics` endpoint, which can then be accessed to receive more detailed monitoring data.

Prometheus is a sophisticated tool to record and log these endpoints, but it requires some configuration first. The reason why it was not used in the end is that the `stats` command offered everything that was required at the moment, and the only change was the editing of the data afterwards to get it into a better format.

Conclusion

The author has written a summary of the history and different functionalities of the Matrix protocol, as well as a selection of its implementations available within the current ecosystem and listed some of the potential competitors Matrix faces on its way to broader adoption.

The Synapse server has been successfully implemented. It operates under the domain `matrix.vse.cz`, with registrations enabled from the Prague University of Economics network. It is able to communicate with other servers worldwide and send and receive messages as well as make and receive calls be they audio or video. Registration to the server also requires an email address, to which the server then sends a confirmation message. The server is able to register users from third-party OIDC providers with users from a Google or Github account being able to login through their existing account. The server's performance over long and short periods under different amounts of load has been documented and analysed, with graphics being made to visualise the collected information.

Possible future improvements were explained, like the implementation of more SSO providers in order to stop storing passwords, more redundancy with multiple instances of the same server running and cooperating at the same time and upgrading to a better SMTP server to not be restricted in which users can register, as well as some more sophisticated ways to monitor the server data.

References

- Adfinis. (2020). *How to set up your own matrix.org homeserver with federation!* Adfinis.com. Retrieved March 20, 2024, from <https://adfinis.com/en/blog/how-to-set-up-your-own-matrix-org-homeserver-with-federation/>
- Albrecht, M. R., Dowling, B., & Jones, D. (2023). *Device-Oriented Group Messaging: A Formal Cryptographic Analysis of Matrix' Core*. Retrieved January 15, 2024, from <https://eprint.iacr.org/2023/1300>
Publication info: Published elsewhere. Major revision. 45th IEEE Symposium on Security and Privacy, S&P 2024 (to appear).
- Boky. (2024). *Bokysan/docker-postfix*. Retrieved April 19, 2024, from <https://github.com/bokysan/docker-postfix>
- Bura, A. (2024). *Cinny*. Retrieved April 27, 2024, from <https://cinny.in>
- Butcher, M. (2016). *Riot wants to be like Slack, but with the flexibility of an underlying open source platform*. TechCrunch. Retrieved January 13, 2024, from <https://techcrunch.com/2016/09/19/riot-wants-to-be-like-slack-but-with-the-flexibility-of-an-underlying-open-source-platform/>
- Computerphile. (2013). *How NOT to Store Passwords! - Computerphile*. Retrieved April 29, 2024, from <https://www.youtube.com/watch?v=8ZtInClXe1Q>
- Daniel Stenberg. (2021). *History of IRC (Internet Relay Chat)*. Retrieved April 29, 2024, from <https://daniel.haxx.se/irchistory.html>
- Dussutour, C. (2021). *French government launches in-house developed messaging service, Tchap — Joinup*. Retrieved April 27, 2024, from <https://joinup.ec.europa.eu/collection/open-source-observatory-osor/document/french-government-launches-house-developed-messaging-service-tchap>
- Element. (2024a). *Element Secure Messenger — About*. Retrieved April 25, 2024, from <https://element.io/about>
- Element. (2024b). *Tchap secure app — French government — Matrix*. Retrieved April 27, 2024, from <https://element.io/case-studies/tchap>
- Ermoshina, K., Musiani, F., & Halpin, H. (2016). End-to-End Encrypted Messaging Protocols: An Overview. In F. Bagnoli, A. Satsiou, I. Stavrakakis, P. Nesi, G. Pacini, Y. Welp, T. Tiropanis & D. DiFranzo (Eds.), *Internet Science* (pp. 244–254). Springer International Publishing. https://doi.org/10.1007/978-3-319-45982-0_22
- faaaaabi. (2023). *A basic setup for synapse with postgres behind a nginx reverse proxy*. Gist. Retrieved January 6, 2024, from <https://gist.github.com/faaaaabi/8042e816f3198e43c611f2a2dcab051c>
- Finley, K. (2014). Out in the Open: Take Back Your Privacy With This Open Source Whats-App [magazine]. *Wired*. Retrieved May 5, 2024, from <https://www.wired.com/2014/05/briar/>
- Imani Rad, A., & Banaeian Far, S. (2023). SocialFi transforms social media: An overview of key technologies, challenges, and opportunities of the future generation of social

- media. *Social Network Analysis and Mining*, 13(1), 42. <https://doi.org/10.1007/s13278-023-01050-7>
- Jake Edge. (2016). *The perils of federated protocols [LWN.net]*. Retrieved May 5, 2024, from <https://lwn.net/Articles/687294/>
- Jamie Juviler. (2024). *What Is an API Endpoint? (And Why Are They So Important?)* Retrieved May 4, 2024, from <https://blog.hubspot.com/website/api-endpoint>
- jeffatrackaid. (2014). *Answer to "How can I disallow access to my website by its IP address?"* Webmasters Stack Exchange. Retrieved March 29, 2024, from <https://webmasters.stackexchange.com/a/67401>
- jevolk. (2023). *Modules/web_hook: Add post-push handler for reacting to push events.* · *matrix-construct/construct@0624b69*. Retrieved May 2, 2024, from <https://github.com/matrix-construct/construct/commit/0624b69246878da592d3f5c2c3737ad0b5ff6277>
- King, B. (2021). *What Is the Matrix Protocol and How Does It Work?* MUO. Retrieved January 13, 2024, from <https://www.makeuseof.com/what-is-matrix-protocol-how-does-it-work/>
- Krille-chan. (2024). *FluffyChat Official Website*. Retrieved April 27, 2024, from <https://fluffychat.im/>
- Lee, D. (2013). Fat Markup: Trimming the Myth one calorie at a time. <https://doi.org/10.4242/BalisageVol10.Lee01>
- Matt Weinberger. (2014). *Matrix wants to smash the walled gardens of messaging*. Computerworld. Retrieved April 22, 2024, from <https://www.computerworld.com/article/1384128/matrix-wants-to-smash-the-walled-gardens-of-messaging.html>
- Merriam-Webster. (2024). *Definition of PROTOCOL*. Retrieved April 14, 2024, from <https://www.merriam-webster.com/dictionary/protocol>
- Metabase. (2024). *Statistiques d'usage de Tchap - Général - Mois* · Metabase. Retrieved April 28, 2024, from https://stats.tchap.incubateur.net/public/dashboard/8c6560e3-c27a-487b-b242-eb1636912f1a?instance=&nom_de_domaine=
- Michael Rogers. (2024). *How it works - Briar*. Retrieved May 5, 2024, from <https://briarproject.org/how-it-works/>
- Pingdom AB. (2017). *IRC is dead, long live IRC*. Retrieved April 29, 2024, from <https://web.archive.org/web/20170815064620/http://royal.pingdom.com/2012/04/24/irc-is-dead-long-live-irc/>
- Sawers, P. (2022). *Inside Matrix, the protocol that might finally make messaging apps interoperable*. TechCrunch. Retrieved April 27, 2024, from <https://techcrunch.com/2022/12/30/inside-matrix-the-protocol-that-might-finally-make-messaging-apps-interoperable/>
- Schipper, G., Seelt, R., & Le-Khac, N.-A. (2021). Forensic analysis of Matrix protocol and Riot.im application. *Forensic Science International: Digital Investigation*, 36. <https://doi.org/10.1016/j.fsidi.2021.301118>
- Simmons, J. (2023). *The future of Synapse and Dendrite*. Retrieved January 4, 2024, from <https://matrix.org/blog/2023/11/06/future-of-synapse-dendrite/>

- simskij. (2023). *Containrrr/watchtower*. Retrieved May 5, 2024, from <https://github.com/containrrr/watchtower>
- Steve Loynes. (2020). *BwMessenger goes live for Bundeswehr!* Element Blog. Retrieved April 28, 2024, from <https://element.io/blog/bwmessenger-goes-live-for-bundeswehr/>
- The Apache Software Foundation. (2023). *Apache HTTP Server Version 2.4 Documentation - Apache HTTP Server Version 2.4*. Retrieved March 29, 2024, from <https://httpd.apache.org/docs/2.4/>
- The Matrix.org Foundation. (2023a). *Elements of Matrix*. Retrieved January 2, 2024, from <https://matrix.org/docs/matrix-concepts/elements-of-matrix/>
- The Matrix.org Foundation. (2023b). *Matrix for Instant Messaging*. Retrieved January 15, 2024, from https://matrix.org/docs/chat_basics/matrix-for-im/
- The Matrix.org Foundation. (2023c). *OpenID Connect - Synapse*. Retrieved May 5, 2024, from <https://element-hq.github.io/synapse/latest/openid.html>
- The Matrix.org Foundation. (2024a). *About Matrix*. Retrieved January 6, 2024, from <https://matrix.org/about/>
- The Matrix.org Foundation. (2024b). *The Matrix Ecosystem*. Retrieved April 26, 2024, from <https://matrix.org/ecosystem/>
- The Matrix.org Foundation. (2024c). *Matrix Specification*. Retrieved April 14, 2024, from <https://spec.matrix.org/v1.10/>
- The Matrix.org Foundation. (2024d). *Matrix-org/dendrite*. Retrieved April 22, 2024, from <https://github.com/matrix-org/dendrite>
- The Matrix.org Foundation. (2024e). *Matrixdotorg/synapse - Docker Image*. Retrieved January 6, 2024, from <https://hub.docker.com/r/matrixdotorg/synapse>
- The Matrix.org Foundation. (2024f). *Spec Change Proposals*. Retrieved April 25, 2024, from <https://spec.matrix.org/unstable/proposals/>
- Thévenet, A. (2023). *BundesMessenger: Shared, reused and interoperable*. — *Joinup*. Retrieved April 28, 2024, from <https://joinup.ec.europa.eu/collection/open-source-observatory-osor/news/bundesmessenger-shared-reused-and-interoperable>
- Thoma, M. (2017). *Answer to "How to clean up Docker"*. Stack Overflow. Retrieved January 6, 2024, from <https://stackoverflow.com/a/45798680>
- Tony Tran. (2022). *How to Automatically Update Docker Container Images with Watchtower on Ubuntu 22.04* — *DigitalOcean*. Retrieved May 5, 2024, from <https://www.digitalocean.com/community/tutorials/how-to-automatically-update-docker-container-images-with-watchtower-on-ubuntu-22-04>
- Veljko Ristić. (2024). *How to Set Up and Configure Gmail SMTP to Send Emails [2024]*. Retrieved April 18, 2024, from <https://mailtrap.io/blog/gmail-smtp/>
- Volpicelli, G. M. (2021). How governments and spies text each other [magazine]. *Wired*. Retrieved April 27, 2024, from <https://www.wired.com/story/matrix-encrypted-messaging-app-governments/>